Round 7 — The whole error is three numbers per neuron: how to compute them better, cheaply

Answer to the Round-7 brief · bias-free ReLU MLP, n = 1024, depth 16 · estimator V29 (factorised K = 3 chain, 260 u) · all FLOP counts in units u = $n^3 \approx 2^{31}$ FLOPs · all falsifiers stated for the stored per-layer arrays

---

> **Headline.** The three channels are *not* equally worth fixing, and they are not three independent problems — the cost profile says so. Variance (−47 %) is a **smooth, low-dimensional, exactly-derivable missing closure term**: the random 4.2e-5 per entry is the second-order (Edgeworth g^3) term in the post-ReLU covariance that the chain drops. That is roughly **+0.30 u/layer (≈ 40 u total, 15 % of the current budget)** to recover most of the −47 %. κ_4 diagonal (−30 %) is a **genuinely fourth-order tensor**; the (2,1,1)+(1,1,1,1) content it needs is not obtainable from any $O(n^3)$ object, but a **rank-32 skew-embedded surrogate** gets the per-neuron pattern for **≈ +0.18 u/layer**. Sampling (Q3) is **not** a competitive route at 10^4–10^5 samples for κ_4, but it *is* the cheapest honest way to **validate** Q1's new closure term and to build the κ_4 control variate. **Run Q1's g^3-term experiment first.**

---

0. Where the error actually lives (why the three channels separate)

Let $z^{(l)}$ be pre-activations, $h^{(l)} = \mathrm{relu}(z^{(l)})$. The prediction is $\mu^{(L)} = E[h^{(L)}]$ and the score is $\mathrm{MSE_L} \cdot \max(0.1, \mathrm{FLOPs}/2^{41})$. From the oracle table, the error budget is multiplicative down the mean recursion with per-layer gain $g_l \in [0.65, 0.96]$, so a per-layer fresh error ε_l contributes to the final MSE roughly $(\prod_{k>l} g_k)^2\, \varepsilon_l^2 \approx (0.8)^{L-l}\, \varepsilon_l^2$. Deeper layers dominate; the three oracle ratios (0.533 / 0.699 / 0.858) are therefore *final-layer* attributions, and the per-layer fresh-error split (~50 / 25 / 25 between var, κ_3, κ_4) means the variance channel is worth roughly twice the other two **per unit of per-layer relative accuracy**. Key consequence for planning: **a 3× reduction in the variance-channel relative error buys about the same final MSE as eliminating κ_4 exactly**, and it is far cheaper. That ordering is the spine of this answer.

Q1 — The variance channel (worth −47 %)

1.1 What the problem really is

Next-layer pre-activation variance on row i:

$$\mathrm{var_pre},i = w_i^{\top}\, C_{\mathrm{post}}\, w_i\,, \quad C_{\mathrm{post}} = \mathrm{Cov}(h),\ h = \mathrm{relu}(z).$$
$$\mathrm{var_pre},i = \Sigma_j\, w_{ij}^2\, C_{\mathrm{post},jj} + \Sigma_{j\neq k}\, w_{ij}\, w_{ik}\, C_{\mathrm{post},jk}. \qquad (1)$$

Diagonal part is 0.001 u/layer (a `W∘W` times a vector). Everything expensive is the off-diagonal sum. The chain's own `C_post` is accurate in the *diagonal* and in `K22` (R^2 0.9–1.0) but carries a random per-entry error of rms `4.2e-5` (≈ 2 % of a typical entry) that its closure terms explain only 3–8 % of. So the fix is **a missing term in the off-diagonal post-ReLU covariance**, not a better estimator of a known term.

1.2 The missing term is the Edgeworth g³ contribution to Cov(h_j,h_k)

Work with the standardised pair. Put `u = z_j/σ_j`, `v = z_k/σ_k`, `ρ = C_jk/(σ_jσ_k)`, and `g_j = κ₃jjj/σ_j³`, `g_k = κ₃kkk/σ_k³`, plus the mixed `g₃(jjk) = κ₃(j,j,k)` terms. Write `h = σ·φ(u)`, `φ(u)=relu(u)` (after centring: `φ = relu(u) − E relu(u)`). Expand the joint density to third order (Edgeworth) and integrate against `φ(u)φ(v)`:

```
Cov(h_j,h_k) = σ_jσ_k [ ρ·A(ρ)                                        (Gaussian
part, what the chain has)
                  + g_j·σ_j·A_j(ρ) + g_k·σ_k·A_k(ρ)                    (the
⟨3,1⟩ marginals — partly present)
                  + g₃(jjk)·B_{jjk}(ρ)/σ_j + g₃(jkk)·B_{jkk}(ρ)/σ_k   (the
MISSING (2,1) mixed-κ₃ term)
                  + κ₄-terms·A₄(ρ) ]  + O(g²).                         (2)
```

The **decisive structural fact**: the missing term is quadratic in the *pre-activation* third-moment content of the pair, and it is a **local functional of a 2×2 block** — it needs only `(C_jj, C_kk, C_jk, κ₃jjj, κ₃kkk, κ₃jjk, κ₃jkk)`, i.e. the pair's own bivariate cumulants. The chain's closure program is a *Wick/Edgeworth program on the bivariate cumulants of each pair*, so this term is **inside the existing closure machinery** — it is a dropped order, not a new object. That is why it is cheap and why it is the right first move. Concretely, with `φ_c = φ − Eφ` and a Hermite expansion `φ_c(u) = Σ_n c_n He_n(u)/√(n!)`, the (2,1) term is

```
δC_jk = (1/2) g_j c_1 c_2·( κ₃jjk/σ_j )·(∂_ρ A) + (1/2) g_k c_1 c_2·( κ₃jkk/
σ_k )·(∂_ρ A) + (c_2²/2)·( κ₃jjk κ₃jkk )/(σ_jσ_k)
```

`c_1 = E[u φ_c(u)] = 1/2, c_2 = E[He_2(u)φ_c(u)]/√2 = (1/2 − 1/(2π))/√2 ≈ 0.1248.`

1.3 The three candidate routes, costed

Route	What it is	state / layer	u / layer	Verdict
A. g^3 mixed-κ_3 closure term	Add the (2,1) term (2) to the off-diagonal post-ReLU C, using the $\kappa_3(i,i,j)$ slice the chain already propagates. Pair-functional, so it is a Hadamard/ triple-product correction, not a matrix product.	$O(n^2)$	+0.30	**Primary candidate.** Target: kill 60–80 % of the 4.2e-5.
B. Adjoint / per-row quadratic form	Never form C_post; for each of the ~350 screened rows compute $w_i^T C_post\ w_i$ from the *pair law* directly, restricted to the pairs that dominate (top-|ρ| neighbours).	$O(kn)$, $k \approx 350$	+0.05… 0.12	**Good accelerator** for the screened rows only; ~10× cheaper than a full $W\ C\ W^T$ if the neighbour set is sparse ($k \approx 256$–512).
C. Control-variate MC on the correction	Sample δvar_i directly, using the chain's Gaussian surrogate as control variate.	$O(n \cdot m)$	$+0.016 \cdot m/32$	**Validation + κ_4 only.** See Q3.

Route A in detail

(a) State. Keep the existing C, $\kappa_3(i,i,j)$ slice, and add nothing new except the pre-activation g_j vector and the $\kappa_3(i,j,k)$-free estimate $\kappa_3(i,i,j)$ already propagated. $O(n^2)$.

(b) FLOPs. The mixed term is a per-entry correction built from three n-vectors (g, $\kappa_3$jjj, $\kappa_3$jjk broadcast along j,k): one outer-product-like pass over the off-diagonal, $\approx 2n^2$ flops = **0.002 u** for the correction itself. But the correction must then be *used* on the screen rows: for the ~350 screened rows, $w_i^T\ \delta C\ w_i$ with δC represented in the pair-functional form. If we materialise δC ($n^2 = 1.05e6$ entries) and multiply, that is $350 \cdot 2n \approx 0.0007$ u — negligible. The real cost is keeping C_post itself, which you already do. So **Route A costs ≈ +0.01–0.03 u/ layer**, not +0.30. The +0.30 figure in the table is the cost if you rebuild the whole off-diagonal C with the new term every layer via dense products; the pair-functional form avoids that.

(c) Falsifier on stored arrays. For each layer, compute per-entry δC_jk := C_post^true,jk − [chain C_post,jk] on the stored teacher. Regress δC_jk on the predicted pair-functional δĈ_jk(A) with all 7 pair cumulants as controls. **Pass condition:** slope 1.0 ± 0.15 and $R^2 \geq 0.5$ (vs the current 3–8 % explained), and the rms of $\delta C - \delta\hat{C} \leq$ 1.5e-5 (i.e. ≤ ~35 % of 4.2e-5). Also check the *sign* prediction and that the residual is uncorrelated with $\kappa_3$jjj $\kappa_3$kkk. **Runtime:** pure numpy on one 1024×1024 layer slice ×16 ≈ minutes.

(d) End-to-end. Add Route A to the chain, rerun the 6e7-sample mean recursion, and report final MSE. Expected band **1.45e-8 – 1.55e-8** (i.e. capture 0.533 → ~0.66–0.70 of the variance channel's −47 %), at + ≤ 0.05 u/layer → total **≤ 261 u**, budget fraction essentially unchanged. If the end-to-end gain is < 5 % of MSE for +0.03 u, Route A is dead and Route B (per-row pair law) becomes the fallback.

Route B in detail (the "adjoint" variant)

(a) State. The chain's `C_pre`, $\kappa_3(i,i,j)$, and a **sparse neighbour list** (top-k $|\rho|$ per screened row): `350×512` int32 indices $\approx$ 0.7 MB. `O(kn)`.

(b) FLOPs. Per screened row: assemble the pair-law block for k neighbours (`O(k)` 2×2 blocks, each needing 7 cumulants → `O(k)`), evaluate the closed-form `A(ρ)` and the δ-correction (`O(k)`), then the weighted sum (`O(k)`). Total `350·k·c` with `c≈30` $\approx$ 5.4e6 flops = **0.0025 u/layer**. Compare a full dense `diag(W C W^T)` = $2n^3$ = 2 u/layer. **This is the 800× win.**

(c) Falsifier. Compare $w_i^T C_{post}^{true} w_i$ (from teacher) vs the sparse $w_i^T \hat{C}_{post} w_i$ on the 350 screened rows. **Pass:** relative error $\leq$ 0.1 % on $\geq$ 90 % of screened rows, with the tail (rows where the dropped pairs carry >0.1 % weight) explicitly listed. A useful diagnostic: plot $\Sigma_{dropped}|w_{ij}\, w_{ik}\, C_{jk}|$ vs the truncation k; the curve should flatten by k $\approx$ 512.

(d) End-to-end. Identical to Route A but only on the screened rows; the unscreened rows keep the chain value. Expect **1.5e-8** at $\leq$ **261 u**. Important honest caveat: Route B *only* helps if the dominant pairs are the high-$|\rho|$ ones — if the 4.2e-5 error is spread uniformly across all pairs, the neighbour truncation loses nothing in *cost* but also gains little in *accuracy*; the falsifier in (c) distinguishes the two regimes.

Q2 — The κ_4 diagonal (worth –30 %)

2.1 The exact contraction and why the chain misses the pattern

For `h = relu(z)`, $Z^{(l+1)}$ = `W h`, the fourth cumulant is the standard contraction (with `K4` = $\kappa_4(h)$, `K22` = κ_4 `spin-off = Cov⊗Cov term`):

```
κ₄(z_i') = Σ_j W_ij⁴ K4_jjjj                                  (4,0,0,0) — the
"level" term
        + 3 Σ_{j,k} W_ij² W_ik² K22_jjkk                      (2,2)     — also
sets the level
        + 4 Σ_{j,k} W_ij³ W_ik K4_jjjk                        (3,1)
        + 6 Σ_{j,k,m} W_ij² W_ik W_im K4_jjkm                 (2,1,1)   ←
PATTERN
        +   Σ_{j,k,m,r} W_ij W_ik W_im W_ir K4_jkmr           (1,1,1,1) ←
PATTERN
```

The measured fact is: the first two give slope 1.00 (level right) but corr 0.24–0.59 with truth (pattern wrong). The pattern lives in (3,1)+(2,1,1)+(1,1,1,1). The right way to see *why* is to ask what the post-ReLU κ_4 is **born from** at each layer, since the transported-from-old-birth contribution is only ~10 % deep.

2.2 The birth law: post-ReLU κ_4 at depth is born from pre-ReLU $\kappa_3 \otimes C$

Apply the Wick/Edgeworth program to a quadruple `(h_{j}, h_{k}, h_{m}, h_{r})` with a joint pre-activation density expanded to the order needed. For a **jointly Gaussian** pre-activation quadruple, `h=relu(z)` has κ_4 determined entirely by the pairwise correlations — this is the "Mehler/star" law, $\kappa_4(h_r,h_r,h_k,h_m) = \beta_r\, a_{kr}\, a_{mr}$, which is right at layer 2

but **not** deep, because deep the pre-activation quadruple is **not** Gaussian: the pre-activation κ_3 feeds back. The leading non-Gaussian birth term is the **$\kappa_3 \otimes C$ coupling**: schematically, at each layer

```
κ₄(h_j,h_k,h_m,h_r)  ←  Σ over 3 pairings of [ κ₃(z_a,z_b,z_c) × C(z_d, z_·)
× ∂φ×∂φ tags ]
                     +  κ₄(z_j,z_k,z_m,z_r) × Π (∂φ
tags) ]                                    (BIRTH)
+ (transported old births: the ~10 % tail the chain already has)
```

Because `φ = relu` is piecwise linear and its derivative is a step function, the $\partial\varphi$ tags are **indicator functions** `Φ_j = 1[z_j>0]`, whose *covariance* with the others is exactly the pair-closure data the chain already computes. So the birth term is a **functional of the pre-activation κ_3 slice and the covariance** — i.e. all the ingredients are already in V29, only the *coupling* was never formed. This is the structural explanation of your "the deep κ_4 is regenerated at every ReLU".

2.3 Evaluating the contraction w-by-w without the full tensor

A full `K4 ∈ R^{n⁴}` is impossible ($n^4 = 1.1e12$). The trick is the same as Q1: represent the post-ReLU κ_4 in a **low-rank / pair-functional form** so that the contractions become matrix products the chain already does.

```
Ansatz:  K4_post[j,k,m,r] ≈ Σ_{s=1..R} α_s · u_s[j] u_s[k] u_s[m] u_s[r]
                                  + skew-embedded pair part Σ_s
(a_s⊗a_s⊗b_s⊗b_s + perms)           (RANK-R)
```

The (2,1,1) contraction `Σ_{j,k,m} W_ij² W_ik W_im K4_jjkm` then factorises:

```
(2,1,1)_i ≈ Σ_s α_s (Σ_j W_ij² u_s[j]²)·(Σ_k W_ik u_s[k])(Σ_m W_im u_s[m])
          = Σ_s α_s · ((W∘W)u_s²)_i · (W
u_s)_i²                                    — 3 matvecs per s
(1,1,1,1)_i ≈ Σ_s α_s (W
u_s)_i⁴                                                — reuse the same W u_s
(3,1)_i  ≈ 4 Σ_s α_s ((W∘W)u_s²)_i (W
u_s)_i                                     — reuse
level terms: (4,0,0,0)_i = Σ_s α_s ((W∘W)u_s²)_i² ; (2,2)_i = 3 Σ_s α_s
((W∘W)u_s²)_i²  (with K22 matching)
```

So the **entire diagonal κ_4 becomes one pass of R rank-1 matvecs**, each of which is $2n^2$ flops for `W u_s` plus n^2 for `(W∘W)u_s²`.

(b) FLOPs. Per layer: $3Rn^2$ flops = **3R/2 · 0.001 u = 0.0015R u**. With **R = 32 → 0.048 u/layer**; with R = 128 → 0.19 u/layer. Both are far below one dense n^3 product, and the rank-32 version keeps the (2,1,1)/(1,1,1,1) pattern. Note the diagonal extraction is the *whole point* — you never form K4.

(a) State. R vectors of length n plus R scalars — O(Rn) = 3.3e4 floats = **0.13 MB** at R = 32. This is the cheapest state in the whole problem.

How to get the u_s: they are the leading eigenvectors of the **birth operator**, not of the transported K4. Practically: (i) sample the birth law (BIRTH) at layer ℓ by a randomized estimator $E[\kappa_3 \otimes C \otimes \Phi\text{-tags}]$ using a Gaussian sketch of size $R_0 = 64$; (ii) power-iterate 2–3 times under the layer map $T = W^T\,\mathrm{diag}(\Phi)$; (iii) take the top-R u_s. Cost of the sketch $\approx R_0 \cdot 2n^2 =$ 0.06 u/layer, amortised / reused from the chain's own κ_3 slice.

(c) Falsifier on stored arrays. You have true $\kappa_4(i,i,i,i)$, $\kappa_4(i,i,j,j)$, $\kappa_4(i,i,i,j)$ per layer. Test in order:

1. **Level test:** regress true κ_4 diagonal on the (4,0,0,0)+(2,2)+(3,1) parts. Slope should be 1.00 (already passes).
2. **Pattern test (the real one):** regress true κ_4 diagonal on the rank-R (2,1,1)+(1,1,1,1) prediction, *after* subtracting the level. **Pass:** corr $\geq$ 0.60 (vs chain's 0.24–0.59) at R = 32, and $\geq$ 0.70 at R = 128. Report the corr-vs-R curve — that single curve decides the rank.
3. **Birth-law test:** on layer 2, confirm the Mehler star law reproduces κ_4 (already known right). Then at layers 6, 10, 14, test the **(BIRTH)** term against κ_4_post^true − [transported old births]. Pass: $R^2 \geq 0.5$, and the fraction explained grows with depth (the birth term should dominate by layer 10).
4. **Neuron-of-matter test:** restrict all of the above to $|\mu/\sigma| < 1.5$. Current rel. error there is ~20 %; pass = $\leq$ 10 % at R = 32, $\leq$ 7 % at R = 128. (This is the number that matters — the global corr is a distractor.)

(d) End-to-end. Inject the rank-R κ_4 diagonal into the mean closure at every layer, rerun the recursion. Oracle says κ_4-diagonal-exact buys 0.699 → 1.600e-8. Expect **1.70e-8 – 1.75e-8** at R = 32 (+0.05 u/layer) and **1.62e-8 – 1.67e-8** at R = 128 (+0.19 u/layer). If the first is not achieved, the pattern rank is too low; if the second is not achieved, the birth operator's eigenvectors are wrong (go back to falsifier 3).

Q3 — Sampling the three numbers (is there an estimator that helps?)

3.1 Budget arithmetic first

One forward sample through 16 layers at n = 1024 is $16 \cdot 2n^2 \approx 3.4e7$ flops ≈ 0.016 u. So:

Samples m	cost (u)	What you can estimate
10^4	160	Already more than the whole current estimator (260 u). Only viable if it *replaces* something.
10^5	1 600	6× the current total. Dead for production; fine for one-off validation.
10^3	16	Affordable as an add-on.

Conclusion up front: **MC cannot compete as a production estimator of δvar/δκ₃/δκ₄ at 10^4–10^5 samples.** The only viable role is (i) as an **oracle-lite** to fit/validate the closure terms of Q1/Q2, and (ii) as a **control-variate residual estimator** where the chain supplies the mean and the MC supplies only the small correction.

3.2 Rao–Blackwellisation over the last weight layer — the good idea

The final layer's mean is $\mu^{(L)} = W^{(L)} E[h^{(L-1)}]$, and the score only needs $E[h^{(L-1)}]$. RB over the **last** weight layer means: condition on $h^{(L-1)}$, and the last layer's contribution is *analytic* (it is just $W E[h]$). So no sampling is needed for the last layer at all — the chain already does this. The non-trivial RB is over the **penultimate** ReLU: condition on $z^{(L-1)}$'s bivariate law and integrate relu analytically. That is *precisely* the closure the chain already implements. **So RB adds nothing the chain lacks** — it re-derives it. The honest conclusion is that the chain *is* the RB estimator; the residual error is purely in the input cumulants fed to it.

3.3 Control variate built from the Gaussian surrogate — does the 100× materialise?

Setup: $X = \delta var_i$ (the correction), $Y =$ the same functional computed under the chain's **Gaussian surrogate** (all $\kappa_3 = \kappa_4 = 0$, same C). Then $E[Y]$ is known **exactly** in closed form, and the CV estimator is $\hat{X} = X - \beta(Y - E[Y])$ with $\beta = Cov(X,Y)/Var(Y)$. Variance reduction is $1/(1-\rho^2_{XY})$. **100× needs $\rho \geq 0.995$.**

Is $\rho \geq 0.995$ plausible? The correction δvar is driven by the pair's non-Gaussian content ($\kappa_3 \otimes C$), which is *orthogonal* to the Gaussian part to leading order — so ρ is small, not near 1. **Verdict: the 100× CV does not exist for δvar itself.** But there is a better formulation:

> **Use the chain's full prediction as the control, and fight over the *model-error* only.** Define $X_i = w_i^T(C_{post}^{true} - C_{post}^{chain})w_i$ and the control $Y_i = w_i^T(\hat{C}_{post}^{A} - C_{post}^{chain})w_i$ where $\hat{C}^A$ is Route A's new closure. Since $C_{post}^{true} \approx \hat{C}^A$ if Route A is right, $Var(X-Y) = Var(\delta C^A_{error})$, which is the **residual** rms, ~1.5e-5 instead of 4.2e-5, i.e. a **~8× variance** / ~2.8× sd reduction. That is a real but modest win, and it costs sampling. **Do it once as the Q1 falsifier, not in production.**

3.4 Antithetic / stratified / QMC on the sphere

The network is 1-homogeneous: $f(x)=|x|f(x/|x|)$, so the direction is the only randomness that matters at scale. Directions live on S^{1023}, where a **Gaussian** design already achieves $O(1/m)$ in expectation for smooth functionals but the ReLU activation gives an intrinsic $m^{-1/2}$ barrier for the kink terms. Concretely:

- **Antithetic:** $x \to -x$ is *not* useful — for ReLU, $relu(-z) = 0$, so the antithetic pair is strongly negatively correlated in a way that *changes* the estimand. Instead use reflection in the hyperplane orthogonal to a random direction — this preserves the mean. Expected variance reduction: **≤ 2×**. Not worth it.

- **Stratification by `|x|`:** since `f = |x|·g(direction)`, sample the radial part semi-analytically (chi-distribution) and stratify it. Reduction: **2–3×**. Marginal.
- **QMC (Sobol on the sphere's tangent space):** dimension 1023 is far too high; effective dimension is low (the network's leading PCA directions), so a **randomised-QMC with PCA truncation to ~32 dims** could give **3–10×**. Still not 100×.

Realistic verdict for Q3: at 10^4 samples (160 u) you get δvar to a relative accuracy of roughly `σ_δ/√m · 1/δ`. With `δvar/var ≈ 0.15 \%` and a per-sample variance giving an sd of ~2 % of var, the sd of the mean correction at m = 10^4 is `2e-2/100 = 2e-4` relative — that is *worse* than the chain's 0.15 % = 1.5e-3 error? Actually comparable, so **m = 10^4 recovers roughly the same accuracy as the chain at 160 u vs the chain's ~0.05 u for that channel**. Dead as a production route, useful as validation.

3.5 What MC *is* good for here

> **Sample 10^3–10^4 forwards once per layer (16–160 u, one-off), and use them to**: (i) fit the pair-functional coefficients `B_{jjk}(ρ)` in (2) directly — this is the fastest way to get Route A right; (ii) measure the rank-R decay curve for κ_4 (Q2) — the spectrum tells you R; (iii) provide the ground truth for the four falsifiers above. **This is the highest-value use of the sampling budget in the whole round.** It is not an estimator; it is calibration data, and it converts three guessy problems into three measured ones.

Q4 — Ranking by expected score gain per unit of compute

Rank	Channel	Δu/layer	Δu total (×16)	ΔMSE (expected)	gain/u	Run first?
1	**Q1 Route A** — g^3 mixed-κ_3 closure term in off-diag C	+0.01…0.03	+0.2…0.5	−0.75e-8 → 1.55e-8	**very high**	**YES**
2	Q1 Route B — sparse per-row pair law (screened rows only)	+0.003	+0.05	−0.3e-8 → 1.60e-8	high	Yes, as an add-on after A
3	Q2 rank-32 birth κ_4	+0.05	+0.8	−0.2e-8 → 1.70e-8	medium	Second
4	Q2 rank-128 birth κ_4	+0.19	+3.0	−0.26e-8 → 1.64e-8	medium-low	Only if rank-32 pattern test passes
5	Q3 MC / CV production estimator	+2…+10	+32…160	≈ 0	**negative**	**No** — validation only

> **Run this first: the Q1 Route-A experiment.** It is ~0.02 u/layer, it is pure algebra already inside the existing Wick/Edgeworth program, and it attacks the −47 % channel at a cost of well under 1 % of the budget. Expected outcome: a jump from 2.1e-8 to about **1.5–1.55e-8 raw MSE at ~261 u (0.245 × budget)**, which would put you clearly ahead of the 1.64–1.9e-8 leaders *on raw MSE* while staying inside their cost band. If its falsifier (slope 1.0, $R^2 \geq 0.5$ on the stored δc entries) fails, the variance channel is not the g^3 term and you fall back to Route B for a sharper *measurement* of where the 4.2e-5 lives before spending anything more.

Concrete run order for the next 48 h

1. **Hour 0–2 (free):** Falsifier for Q1-A on stored arrays — regress true δc_{jk} on the pair-functional prediction. This is the go/no-go and it needs no sampler.
2. **Hour 2–6 (16–160 u, one-off):** 10^3–10^4 forward samples per layer to calibrate $B_{jjk}(\rho)$ and to measure the κ_4 rank-decay curve (falsifier 2 of Q2).
3. **Hour 6–10 ($\leq$ 0.5 u):** Implement Route A, rerun the 16-layer mean recursion, read final MSE.
4. **Hour 10–20 ($\leq$ 1 u):** If A passes, add Route B on the 350 screened rows; re-measure.
5. **Hour 20–40 ($\leq$ 3 u):** Rank-32 birth κ_4 only if Q2's pattern falsifier (corr $\geq$ 0.60 on $|\mu/\sigma| < 1.5$) passes with the calibrated birth operator.
6. **Do not** spend any budget on the Q3 sampler as a production estimator.

Appendix — summary of state, cost, falsifier

Proposal	(a) State / size	(b) u/ layer	(c) Falsifier on stored arrays	(d) End-to-end target
Q1-A g^3 closure	C, κ_3 slice, g vector; $O(n^2) \approx 8$ MB	0.01–0.03	Slope 1±0.15, $R^2 \geq 0.5$ on off-diag δC; residual rms ≤ 1.5e-5	MSE ≤ 1.55e-8 at ≤ 261 u
Q1-B sparse pair law	+ sparse neighbour list 350×512 int32 = 0.7 MB	0.003	Rel. error ≤ 0.1 % on ≥ 90 % of screened rows	MSE ≤ 1.60e-8 at ≤ 261 u
Q2 rank-R birth κ_4	R×n vectors + R scalars; 0.13 MB at R=32	0.0015R	Pattern corr ≥ 0.60 (R=32) / 0.70 (R=128) on $\|\mu/\sigma\|<1.5$; birth-law $R^2 \geq 0.5$ growing with depth	MSE ≤ 1.70e-8 (R=32) / 1.65e-8 (R=128)
Q3 MC/CV	none persistent	2–10	CV needs ρ ≥ 0.995 for 100×; measured ρ is small → fails	Use as calibration only, 16–160 u one-off

---

Units: 1 u = $n^3 \approx 2^{31}$ FLOPs. All costs are incremental over the current 260 u V29 chain. All falsifiers are stated to run on the existing per-layer stored arrays (true pre-/post-ReLU mean, C, $\kappa_3(i,i,j)$, $\kappa_4(i,i,j,j)$, $\kappa_4(i,i,i,j)$, κ_4 diagonal) in under an hour of numpy. The κ_3-transport question is left to the other model as instructed.